Static:

Such properties that are not related to the object are called static variable or static methods.

They are not related to the object but are common for all like population of all humans is seven billion.

in order to access the static variable we only use the classname like

```
Humans.population += 1;
```

also inside class constructor.

it does not exist in object. it exist in the class template.

So everytime we made a new object of Human the population is incremented.

So it can be used without creating the object even if object is not made it is still there.

we can make the methods static as well like

```
public static void main() { };
```

Why because we can use (run) main without making object of that class

Restrictions:

we cannot have a non-static variable or method inside a static method like

in main we cannot have a greeting method if it is non-static because non-static belongs to an instance like object of a class but static does not belong to an instance.

But inside a non-static method we can have a static method.

Inside a method we can access a non static method or variable through object like

```
obj.x
```

So inside static we can access the non-static method by . like

obj.x.

Because non-static depends on object but static dont.

Thats why when we make a human class like

Human H1 = new Human();

we do H1.age or something.

One more restriction:

we cannot use this inside static method because this belongs to object like this.age. So we cannot because static belongs to class not the object.

So a problem arise like how do we instantiate them like it something is static inside a class we cannot use this. like if a static method.

```
Static Hello(){
    sout(" this.name);
}
```

but name is non static

it cannot be used inside ~~class~~ method which is ~~no~~ static

first problem solution is:

nesting a static class like

```
class Outer {
    static class Inner{}
        void sayHi(){ }
}
}

main () {
    Outer.Inner obj = new Outer.Inner();
    obj.sayHi();
}
```

Second problem solution is:

we cannot use this So we use class.

```
Myclass.printCount();
```

even inside class.

third problem if something is in static variables and we want to initialize them how we do this is through static block.

```
public class StaticBlocking{

    Static int a=4;
    Static int b;

    static {

        b = a * 5;

    } // it is called when object is created
      // only once

}
```

→ as soon as class is loaded it is run first.

→ it is only run once as the first object is created then not

like

```
Myclass obj = new Myclass(); → here it runs
My class obj 2 = new My class(); → here it not runs
```

Other way is Outer and Inner classes.

in sout

System.out.println();

- System → class
- out → its method
- println() → its out method.

Private means it cannot be used outside that file and class not out side.